


**Ministério da Educação**

**Instituto Técnico Comercial de Luanda**

# DESENVOLVIMENTO DE UM SISTEMA WEB DE ALERTA DE ÁGUA E ENERGIA ELÉCTRICA POR BAIRRO

Desenho e Implementação de um Protótipo Colaborativo Baseado na Arquitectura LAMP para Mitigação de Crises de Abastecimento Urbano

Nome: Mariano João Muondo.

Turma: CTI

N.º 16

**Luanda, Angola**

**2026**

# Sumário

**1. Introdução e Enquadramento**.....4
1.1. Contextualização Histórica e Social.....4
1.2. Justificação Científica e Prática do Tema.....4
1.3. Delimitação do Estudo e Estado do Projecto.....4
**2. Estudo de Caso e Diagnóstico Urbano**.....4
2.1. O Cenário da Região Metropolitana de Luanda.....4
2.2. Fluxo Actual de Informações Informais.....5
2.3. Papel e Limites do Sistema Proposto.....5
**3. Problematização e Pergunta de Investigação**.....5
3.1. Delimitação do Problema.....5
3.2. Impactos Possíveis.....5
3.3. Pergunta de Investigação.....6
**4. Objectivos da Investigação**.....6
4.1. Objectivo Geral.....6
4.2. Objectivos Específicos.....6
**5. Metodologia**.....7
5.1. Abordagem e Etapas.....7
5.2. Estado das Etapas.....7
**6. Fundamentação Teórica e Revisão Bibliográfica**.....8
6.1. Sistemas Web e Arquitectura Cliente-Servidor.....8
6.2. Sistemas de Informação e Engenharia de Requisitos.....8
6.3. Bases de Dados Relacionais.....8
6.4. Sistemas Colaborativos e Informação Geográfica Voluntária.....8
6.5. Segurança de Aplicações Web e Usabilidade.....8
**7. Engenharia de Requisitos do Sistema**.....8
7.1. Requisitos Funcionais (RF).....9
7.2. Requisitos Não Funcionais (RNF).....9
7.3. Utilizadores e Módulos.....10
**8. Estudo de Comparação e Estado da Arte**.....10
8.1. Análise de Soluções Existentes.....10
8.2. Matriz Comparativa.....10
**9. Tecnologias Seleccionadas**.....11
9.1. Front-end: HTML5, CSS3 e JavaScript.....11
9.2. Back-end e Dados: PHP, PDO, MySQL e Apache.....11
9.3. Arquitectura LAMP e Ambiente de Desenvolvimento.....11
**10. Arquitectura e Modelação da Base de Dados**.....12
10.1. Arquitectura Cliente-Servidor em Três Camadas.....12
10.2. Fluxo de Submissão e Consulta de Alertas.....12
10.3. Modelo Lógico Proposto.....12
10.4. Regras de Integridade.....12
**11. Lógica de Consolidação e Validação dos Alertas**.....13
11.1. Níveis de Informação e Estados.....13
11.2. Procedimento Proposto.....13
11.3. Parâmetros e Limitações.....14

**12. Segurança e Protecção dos Dados**.....14
**13. Comparação Crítica das Ferramentas Computacionais**.....14
13.1. MySQL, PostgreSQL e Bases de Dados NoSQL.....14
13.2. PHP Nativo e Frameworks PHP.....15
13.3. CSS, JavaScript e Servidor Web.....15
**14. Plano de Testes do Protótipo**.....15
**15. Conclusões e Considerações Finais**.....16
15.1. Contribuições Principais.....16
15.2. Limitações.....16
15.3. Trabalhos Futuros.....16
**16. Referências Bibliográficas**.....16

# 1. Introdução e Enquadramento

## 1.1. Contextualização Histórica e Social

A urbanização acelerada das grandes cidades coloca desafios importantes à gestão das redes de infra-estrutura básica. O abastecimento de água potável e de energia eléctrica é essencial para a saúde, para a actividade económica e para a dignidade das famílias. Quando o fornecimento é interrompido, os moradores precisam de saber rapidamente se o problema afecta apenas a sua residência ou todo o bairro, para poderem organizar-se (por exemplo, armazenar água, recorrer a um gerador ou adiar uma actividade).

Castells (2011) defende que a sociedade contemporânea se organiza em torno de redes de informação. Neste sentido, a falta de canais organizados para comunicar e consultar ocorrências dificulta a coordenação da comunidade e a tomada de decisões no quotidiano. É neste contexto que se insere o presente trabalho.

## 1.2. Justificação Científica e Prática do Tema

O tema justifica-se pela possibilidade de aplicar conceitos de Engenharia de Software, de bases de dados relacionais e de desenvolvimento web a um problema concreto da vida urbana. Em Luanda, a comunicação de interrupções de água e de energia eléctrica faz-se muitas vezes por chamadas telefónicas, mensagens e publicações em redes sociais, sem estrutura comum e sem registo histórico consultável.

O trabalho propõe um sistema web colaborativo que tem o **bairro** como unidade de organização da informação. A ideia assenta na recolha participativa de dados, em que os próprios cidadãos fornecem informação sobre a sua localização (Goodchild, 2007; Howe, 2008). A intenção é organizar os relatos, reduzir a duplicação e apresentar uma visão consolidada por bairro, apoiando a comunidade no acesso à informação.

## 1.3. Delimitação do Estudo e Estado do Projecto

O estudo limita-se ao contexto urbano de Luanda e à comunicação de interrupções de água e de energia eléctrica. O sistema proposto **não substitui** os sistemas técnicos de monitorização das concessionárias, **não confirma** automaticamente a existência de uma avaria física e **não garante** a reparação do serviço. A sua função é apoiar a circulação de informação entre cidadãos.

Em relação ao estado do projecto, esta versão do trabalho documenta a análise do problema, a especificação de requisitos, a selecção tecnológica, a arquitectura, a modelação da base de dados, a lógica de consolidação e o plano de testes. Não são apresentadas evidências de instalação, publicação, validação por entidades públicas ou ensaio com cidadãos. Por isso, as funcionalidades descritas devem ser entendidas como **especificação e desenho do protótipo**, e os resultados de testes constam como pendentes.

# 2. Estudo de Caso e Diagnóstico Urbano

## 2.1. O Cenário da Região Metropolitana de Luanda

A Região Metropolitana de Luanda é o contexto do estudo de caso. Trata-se de uma área urbana extensa e densamente povoada, com grande diversidade de bairros e uma organização administrativa que dificulta a recolha uniforme de informação local. O abastecimento de água é

da responsabilidade da Empresa Pública de Águas de Luanda (EPAL) e a distribuição de energia eléctrica está associada à Empresa Nacional de Distribuição de Electricidade (ENDE).

Este trabalho não apresenta estatísticas sobre a população, a frequência das interrupções ou os prejuízos económicos, porque não foram recolhidas fontes verificáveis para sustentar esses valores. Estudos futuros poderão incluir dados oficiais, por exemplo do Instituto Nacional de Estatística e das próprias concessionárias.

### 2.2. Fluxo Actual de Informações Informais

Na ausência de um canal comunitário organizado e acessível, os moradores recorrem frequentemente a chamadas telefónicas e a redes sociais (como Facebook, WhatsApp e X) para saber se uma falha é local ou geral. Este processo tem os seguintes problemas:

- **dispersão:** os relatos ficam espalhados por grupos e publicações diferentes;
- **repetição:** a mesma ocorrência é relatada várias vezes por pessoas distintas;
- **desactualização:** não há indicação clara de quando a situação foi resolvida;
- **localização imprecisa:** o local e o tipo de falha (água, energia ou ambas) nem sempre são indicados;
- **ausência de histórico:** a informação não fica registada de forma estruturada para consulta posterior.

Segundo Sommerville (2011), a falta de estrutura na recolha de informação dificulta o seu tratamento e a sua reutilização. Estas limitações justificam a proposta de um sistema que organize os relatos por bairro, tipo de serviço e data.

### 2.3. Papel e Limites do Sistema Proposto

O sistema pretende apoiar os cidadãos a registar ocorrências, a consultar o estado dos alertas por bairro e a perceber se existem vários relatos semelhantes. Um alerta consolidado pela plataforma representa a percepção da comunidade e **não** uma confirmação oficial. Qualquer confirmação por parte da EPAL ou da ENDE exigiria uma fonte própria, que está fora do âmbito deste protótipo.

## 3. Problematização e Pergunta de Investigação

### 3.1. Delimitação do Problema

O problema abordado é de comunicação e organização da informação: os relatos sobre interrupções de água e de energia eléctrica circulam por canais dispersos, sem estrutura comum, sem consolidação e sem histórico. Em consequência, é difícil distinguir uma situação isolada (por exemplo, um problema interno de uma habitação) de uma possível interrupção que afecta vários moradores do mesmo bairro.

### 3.2. Impactos Possíveis

A falta de informação organizada pode dificultar o planeamento das famílias e dos pequenos negócios, que dependem de água e de energia para as suas actividades. Neste trabalho esses impactos são tratados como **hipóteses de motivação** e não como resultados medidos, uma vez que não são apresentados dados verificáveis sobre custos ou prejuízos.

### 3.3. Pergunta de Investigação

Com base no problema exposto, formula-se a seguinte pergunta de investigação:

> **«De que forma o desenvolvimento de um sistema web colaborativo de alertas por bairro pode melhorar a organização, a consulta e a divulgação de informações sobre interrupções no abastecimento de água e energia eléctrica em Luanda?»**

## 4. Objectivos da Investigação

### 4.1. Objectivo Geral

Desenvolver um sistema web colaborativo de alerta de falta de água e energia eléctrica por bairro, no contexto urbano de Luanda, que permita aos cidadãos registar ocorrências, consultar o estado do abastecimento e acompanhar alertas consolidados por localização, contribuindo para uma melhor circulação de informação e para a resposta comunitária às interrupções dos serviços.

O objectivo geral indica a finalidade prática do projecto. O seu alcance real está descrito na secção 1.3 e na secção 5.2.

### 4.2. Objectivos Específicos

Para concretizar o objectivo geral, definem-se as seguintes etapas:

1. Realizar uma revisão bibliográfica sobre sistemas web, sistemas de informação, engenharia de software, bases de dados relacionais, sistemas colaborativos e tecnologias de informação aplicadas à comunicação de ocorrências urbanas.
2. Identificar e especificar os requisitos funcionais (RF) e os requisitos não funcionais (RNF) do sistema, considerando as funcionalidades, a segurança, a privacidade, a usabilidade, a compatibilidade e o desempenho.

   **Requisitos funcionais (RF):** RF-001 Registo de utilizadores; RF-002 Autenticação; RF-003 Gestão de perfil; RF-004 Selecção de bairro; RF-005 Submissão de ocorrências; RF-006 Registo temporal; RF-007 Consulta de alertas; RF-008 Painel público; RF-009 Consolidação de ocorrências; RF-010 Prevenção de duplicações; RF-011 Gestão de estados; RF-012 Relatórios; RF-013 Registo de alterações.

   **Requisitos não funcionais (RNF):** RNF-001 Segurança; RNF-002 Privacidade; RNF-003 Integridade; RNF-004 Usabilidade; RNF-005 Responsividade; RNF-006 Compatibilidade; RNF-007 Desempenho; RNF-008 Manutenção; RNF-009 Cópias de segurança; RNF-010 Disponibilidade. A descrição detalhada consta no capítulo 7.
3. Analisar o problema da circulação de informações sobre interrupções no abastecimento de água e energia eléctrica e definir os utilizadores, os módulos e os processos que o sistema deverá apoiar.
4. Modelar uma base de dados relacional para armazenar utilizadores, bairros, tipos de ocorrência, alertas, datas, estados e registos de validação.
5. Comparar e justificar a selecção das tecnologias de desenvolvimento, incluindo HTML5, CSS3, JavaScript, PHP, MySQL e Apache.

6. Desenhar e desenvolver as interfaces e funcionalidades do sistema web, incluindo registo de utilizadores, autenticação, submissão de ocorrências, consulta de alertas por bairro e apresentação do estado do abastecimento.
7. Definir e implementar, conforme o estado real do protótipo, um mecanismo de consolidação de ocorrências que reduza a duplicação de relatos e ajude a distinguir situações isoladas de possíveis interrupções colectivas.
8. Definir e, quando existir um protótipo executável, realizar testes funcionais, de validação de entradas, de segurança básica e de usabilidade, utilizando dados fictícios quando não houver autorização para utilizar dados reais.

Nesta versão do trabalho, os objectivos 1 a 5 e 8 (definição do plano) estão documentados; os objectivos 6, 7 e a execução dos testes dependem da implementação do protótipo e constam como etapas a concluir.

# 5. Metodologia

## 5.1. Abordagem e Etapas

O trabalho segue uma abordagem de investigação aplicada, de natureza qualitativa e orientada para o desenvolvimento de um artefacto de software. As etapas são as seguintes:

- **pesquisa bibliográfica** em livros de engenharia de software, bases de dados e desenvolvimento web, e em documentação técnica oficial;
- **análise do problema** com base na observação do contexto e na literatura consultada;
- **levantamento e especificação de requisitos** funcionais e não funcionais;
- **modelação** da arquitectura e da base de dados;
- **selecção das tecnologias** com base em critérios de adequação ao âmbito, custo e simplicidade;
- **desenho das interfaces** e **desenvolvimento** das funcionalidades;
- **testes** funcionais, de segurança básica e de usabilidade, com dados fictícios.

Não foram realizadas entrevistas, inquéritos, visitas de campo ou recolhas de dados com utilizadores reais. A análise do contexto baseia-se na literatura e na experiência geral do problema, e esta limitação é assumida.

## 5.2. Estado das Etapas

**Tabela 1 – Estado das etapas do trabalho**

| Etapa | Estado | Observação |
|---|---|---|
| Pesquisa bibliográfica | Concluída (nesta versão) | Fontes listadas no capítulo 16; a edição de algumas referências deve ser confirmada no exemplar consultado. |
| Análise do problema | Concluída | Baseada na literatura e no contexto descrito. |
| Requisitos (RF e RNF) | Concluída | Capítulo 7. |
| Modelação (arquitectura e dados) | Concluída (modelo proposto) | Capítulo 10; não existe evidência de base de dados instalada. |
| Selecção tecnológica | Concluída | Capítulos 9 e 13. |
| Desenvolvimento do protótipo | Sem evidência anexada | Código-fonte não anexado a esta versão; funcionalidades tratadas como especificadas. |

| Etapa | Estado | Observação |
|---|---|---|
| Testes | Pendentes | Plano no capítulo 14; resultados não executados. |

# 6. Fundamentação Teórica e Revisão Bibliográfica

## 6.1. Sistemas Web e Arquitectura Cliente-Servidor

Um sistema web funciona segundo o modelo cliente-servidor: o navegador (cliente) envia pedidos HTTP a um servidor, que processa a lógica da aplicação, consulta a base de dados e devolve uma resposta, normalmente em HTML (Sommerville, 2011). A separação em camadas de apresentação, lógica de negócio e dados facilita a manutenção e a evolução do sistema.

## 6.2. Sistemas de Informação e Engenharia de Requisitos

Um sistema de informação recolhe, armazena, processa e disponibiliza dados para apoiar decisões. A engenharia de requisitos permite identificar o que o sistema deve fazer (requisitos funcionais) e com que qualidades deve fazê-lo (requisitos não funcionais), de forma verificável e rastreável (Pressman & Maxim, 2016; Sommerville, 2011).

## 6.3. Bases de Dados Relacionais

O modelo relacional organiza os dados em tabelas ligadas por chaves primárias e estrangeiras, e garante a integridade através de restrições e de transacções com propriedades ACID (Elmasri & Navathe, 2011). A normalização reduz a redundância e as anomalias de actualização.

## 6.4. Sistemas Colaborativos e Informação Geográfica Voluntária

Nos sistemas colaborativos, os próprios utilizadores contribuem com informação (Howe, 2008). Quando essa informação tem componente geográfica, fala-se em informação geográfica voluntária, em que os cidadãos funcionam como «sensores» (Goodchild, 2007). Plataformas como o Ushahidi aplicam este princípio à comunicação de ocorrências. O principal desafio é a fiabilidade da informação, que exige validação e consolidação.

## 6.5. Segurança de Aplicações Web e Usabilidade

A OWASP (2021) identifica os principais riscos de segurança de aplicações web, entre os quais a injecção de código e as falhas de controlo de acesso e de autenticação. Em PHP, recomenda-se o uso de consultas parametrizadas com PDO e de funções próprias para o hashing de palavras-passe (PHP Group, s.d.). Quanto à interface, o HTML5 oferece elementos semânticos que melhoram a acessibilidade (W3C, 2014) e as interfaces responsivas adaptam-se a diferentes tamanhos de ecrã.

# 7. Engenharia de Requisitos do Sistema

Os requisitos funcionais definem as operações que o sistema deve realizar e os não funcionais definem as qualidades exigidas (Pressman & Maxim, 2016). Cada requisito tem um código e está ligado aos objectivos específicos (OE). A coluna «Estado» usa três categorias: **Núcleo** (previsto no âmbito do protótipo), **Complementar** (desejável, se o tempo o permitir) e **Trabalho futuro**. Nenhum requisito é apresentado como já implementado.

## 7.1. Requisitos Funcionais (RF)

**Tabela 2 – Requisitos funcionais**

| Código | Requisito | Descrição | Estado | OE |
|---|---|---|---|---|
| RF-001 | Registo de utilizadores | Criação de conta com os dados estritamente necessários (nome, e-mail único, palavra-passe e bairro). | Núcleo | OE 2, 6 |
| RF-002 | Autenticação | Entrada e encerramento de sessão de utilizadores registados. | Núcleo | OE 6 |
| RF-003 | Gestão de perfil | Consulta e actualização autorizada dos dados do utilizador. | Complementar | OE 6 |
| RF-004 | Selecção de bairro | Associação dos registos a um bairro previamente definido numa lista. | Núcleo | OE 4, 6 |
| RF-005 | Submissão de ocorrências | Comunicação de falta de água, de energia eléctrica ou de ambas. | Núcleo | OE 6 |
| RF-006 | Registo temporal | Guardar a data e a hora de cada ocorrência. | Núcleo | OE 4 |
| RF-007 | Consulta de alertas | Consulta por bairro, tipo de serviço, data e estado. | Núcleo | OE 6 |
| RF-008 | Painel público | Apresentação clara dos estados dos alertas, sem divulgar dados pessoais. | Núcleo | OE 6 |
| RF-009 | Consolidação de ocorrências | Agregação de relatos semelhantes por período, bairro e tipo de serviço. | Núcleo | OE 7 |
| RF-010 | Prevenção de duplicações | Limitar ou sinalizar relatos repetidos do mesmo utilizador para a mesma ocorrência. | Núcleo | OE 7 |
| RF-011 | Gestão de estados | Actualização controlada do estado de um alerta (ver secção 11.1). | Complementar | OE 7 |
| RF-012 | Relatórios | Resumos ou estatísticas das ocorrências registadas. | Trabalho futuro | OE 3 |
| RF-013 | Registo de alterações | Histórico das alterações relevantes e das operações administrativas. | Complementar | OE 4, 8 |

## 7.2. Requisitos Não Funcionais (RNF)

Os valores de desempenho e de disponibilidade são metas de projecto e não resultados medidos.

**Tabela 3 – Requisitos não funcionais**

| Código | Dimensão | Descrição | Forma de verificação |
|---|---|---|---|
| RNF-001 | Segurança | Proteger contas, sessões e operações (hashing de palavras-passe, consultas parametrizadas, controlo de acesso). | Verificação por testes de segurança básica (T-08 a T-10). |
| RNF-002 | Privacidade | Recolher o mínimo de dados pessoais e não os divulgar no painel público. | Revisão do painel público e dos dados armazenados. |
| RNF-003 | Integridade | Preservar a consistência dos registos através de chaves estrangeiras e restrições. | Tentativa de inserção inválida deve ser rejeitada. |
| RNF-004 | Usabilidade | Interface compreensível, com submissão de ocorrência em poucos passos. | Teste com utilizadores fictícios ou voluntários (T-12). |
| RNF-005 | Responsividade | Adaptação a telemóveis, tablets e computadores. | Verificação em larguras de ecrã diferentes. |
| RNF-006 | Compatibilidade | Funcionamento nas versões recentes do Chrome, Firefox, Edge e Safari. | Teste manual em cada navegador. |

| Código | Dimensão | Descrição | Forma de verificação |
|---|---|---|---|
| RNF-007 | Desempenho | Meta de projecto: carregar o painel público em cerca de 3 segundos em ligação móvel típica. | Medição com as ferramentas do navegador; valor ainda não medido. |
| RNF-008 | Manutenção | Código organizado por camadas e comentado, com configuração separada. | Revisão do código. |
| RNF-009 | Cópias de segurança | Procedimento periódico de cópia e recuperação da base de dados. | Teste de restauro. |
| RNF-010 | Disponibilidade | Meta dependente da infra-estrutura de alojamento escolhida; não se fixa percentagem sem fundamentação. | A definir após a escolha do alojamento. |

### 7.3. Utilizadores e Módulos

O sistema considera três perfis: **visitante** (consulta o painel público), **utilizador registado** (submete ocorrências e consulta alertas) e **administrador** (gere bairros, estados e parâmetros). Os módulos principais são: autenticação, ocorrências, consolidação de alertas, painel público e administração.

## 8. Estudo de Comparação e Estado da Arte

### 8.1. Análise de Soluções Existentes

As soluções para comunicação de ocorrências dividem-se em canais directos (chamadas telefónicas), redes sociais e plataformas colaborativas específicas. O Ushahidi é uma plataforma de código aberto para recolha e mapeamento de relatos de cidadãos, usada em situações de crise, e serve de referência conceptual. Não foi feita aqui uma avaliação técnica detalhada dessa plataforma nem das aplicações das concessionárias locais, pelo que a comparação seguinte se limita a características gerais e a **vantagens teóricas** do protótipo.

### 8.2. Matriz Comparativa

**Tabela 4 – Comparação entre as soluções**

| Critério | Canais telefónicos | Redes sociais | Protótipo proposto (previsto) |
|---|---|---|---|
| Organização dos dados | Registo próprio do operador; pode não ser público | Mensagens dispersas | Dados estruturados em base de dados relacional |
| Facilidade de consulta | Depende do atendimento | Exige pesquisa manual | Painel por bairro |
| Identificação geográfica | Depende do relato oral | Frequentemente omitida | Associada ao bairro seleccionado |
| Registo histórico | Depende da entidade | Difícil de reunir | Previsto (data, tipo, estado) |
| Consolidação de ocorrências | Não aplicável | Não existe de forma nativa | Prevista (secção 11) |
| Facilidade de utilização | Simples, mas sujeita a filas | Conhecida pelos utilizadores | A validar com utilizadores |
| Custo de implementação e manutenção | Custos da entidade | Sem custo directo | Custo de alojamento e manutenção |
| Segurança e controlo de acesso | Controlado pela entidade | Controlado pela rede social | Previsto (secção 12) |
| Dependência da Internet | Não depende | Depende | Depende |
| Actualização dos alertas | Depende do operador | Depende dos utilizadores | Prevista (expiração e resolução) |

*Fonte: elaboração própria, com base em características gerais; sem testes comparativos.*

O protótipo não é considerado superior em todos os critérios: depende de ligação à Internet, precisa de adesão da comunidade e a sua fiabilidade ainda não foi demonstrada.

## 9. Tecnologias Seleccionadas

Os componentes seleccionados são compatíveis numa arquitectura web convencional. A utilização efectiva de cada um deve ser confirmada no código-fonte; enquanto este não for anexado, são descritos como **componentes propostos**.

### 9.1. Front-end: HTML5, CSS3 e JavaScript

- **HTML5:** define a estrutura das páginas com elementos semânticos (`header`, `nav`, `section`, `article`), o que favorece a acessibilidade (W3C, 2014).
- **CSS3:** define a apresentação visual; propõe-se o uso de Flexbox, CSS Grid e Media Queries para obter uma interface responsiva, partindo do ecrã de telemóvel (abordagem mobile-first).
- **JavaScript:** proporciona interactividade, validação prévia dos formulários no navegador e envio assíncrono de dados através da Fetch API (MDN Web Docs, s.d.). A validação no navegador não substitui a validação no servidor.

### 9.2. Back-end e Dados: PHP, PDO, MySQL e Apache

- **Apache:** servidor web que recebe os pedidos HTTP e os encaminha para o PHP (Apache Software Foundation, s.d.).
- **PHP:** processa a lógica do lado do servidor, gere sessões, valida dados e aplica as regras de consolidação.
- **PDO:** interface de acesso à base de dados que suporta consultas parametrizadas (PHP Group, s.d.).
- **MySQL:** sistema de gestão de bases de dados relacionais (Oracle, s.d.).

### 9.3. Arquitectura LAMP e Ambiente de Desenvolvimento

LAMP significa Linux, Apache, MySQL e PHP. O Linux é o sistema operativo típico de um servidor de produção. Em ambiente local no Windows, o equivalente é o WAMP (por exemplo, o WampServer), que reúne Apache, MySQL e PHP. Assim, **só se pode afirmar que o sistema usa LAMP se for executado em Linux**; se o desenvolvimento ocorrer apenas em Windows, trata-se de um ambiente WAMP, compatível com a mesma pilha de software. O phpMyAdmin pode ser usado para administrar a base de dados, e o Visual Studio Code como editor de código, se for efectivamente utilizado.

**Tabela 5 – Funções e comunicação dos componentes**

| Componente | Função | Comunica com |
|---|---|---|
| Navegador (HTML5, CSS3, JS) | Apresentação e interacção | Servidor Apache via HTTP/HTTPS |
| Apache | Recebe pedidos e serve ficheiros | PHP |
| PHP | Lógica de negócio e sessões | MySQL via PDO |
| MySQL | Persistência dos dados | PHP |
| Linux / WAMP | Sistema onde corre o servidor | Todos os componentes do servidor |

# 10. Arquitectura e Modelação da Base de Dados

## 10.1. Arquitectura Cliente-Servidor em Três Camadas

Propõe-se uma arquitectura em três camadas: **apresentação** (HTML5, CSS3 e JavaScript), **lógica de negócio** (PHP) e **persistência** (MySQL). A camada de apresentação comunica com a de negócio por HTTP, e apenas esta acede à base de dados. Esta separação facilita a manutenção e permite alterar uma camada com impacto limitado nas outras (Sommerville, 2011).

## 10.2. Fluxo de Submissão e Consulta de Alertas

1. O utilizador autenticado selecciona o tipo de serviço e submete a ocorrência no navegador.
2. O navegador valida o preenchimento e envia o pedido ao servidor.
3. O PHP valida os dados, verifica a sessão e as regras de duplicação.
4. Se válida, a ocorrência é gravada na base de dados e o módulo de consolidação reavalia o alerta do bairro.
5. O painel público consulta os alertas e apresenta o estado por bairro, sem dados pessoais.

## 10.3. Modelo Lógico Proposto

O modelo abaixo é **proposto** a partir dos requisitos; não corresponde a uma base de dados comprovadamente instalada. Procura evitar redundâncias, em linha com os princípios de normalização (Elmasri & Navathe, 2011); a verificação formal de uma forma normal específica exigiria análise adicional das dependências funcionais.

**Tabela 6 – Entidades, atributos e chaves**

| Entidade | Atributos principais (PK sublinhada na descrição) | Relações (FK) |
|---|---|---|
| municipio | id_municipio (PK), nome | — |
| bairro | id_bairro (PK), nome, id_municipio | id_municipio → municipio |
| utilizador | id_utilizador (PK), nome, email (único), palavra_passe_hash, perfil, data_registo, id_bairro | id_bairro → bairro |
| tipo_servico | id_tipo (PK), descricao (água, energia eléctrica) | — |
| estado_alerta | id_estado (PK), descricao | — |
| alerta | id_alerta (PK), id_bairro, id_tipo, id_estado, inicio, ultima_actualizacao, total_utilizadores | bairro, tipo_servico, estado_alerta |
| ocorrencia | id_ocorrencia (PK), id_utilizador, id_bairro, id_tipo, data_hora, id_alerta (pode ser nulo) | utilizador, bairro, tipo_servico, alerta |
| parametro_sistema | chave (PK), valor (janela em minutos, limiar de relatos) | — |
| registo_auditoria | id_registo (PK), id_utilizador, accao, data_hora | utilizador |

Uma comunicação de «água e energia» é guardada como duas ocorrências (uma por serviço), o que evita relações muitas-para-muitas desnecessárias.

## 10.4. Regras de Integridade

- chaves primárias em todas as tabelas e chaves estrangeiras para as relações indicadas;
- restrição de unicidade no e-mail do utilizador;

- campos obrigatórios (`NOT NULL`) para bairro, tipo de serviço e data/hora da ocorrência;
- estados limitados aos valores da tabela `estado_alerta`;
- regra de duplicação: um mesmo utilizador não pode registar nova ocorrência do mesmo tipo e no mesmo bairro dentro da janela de agrupamento.

# 11. Lógica de Consolidação e Validação dos Alertas

## 11.1. Níveis de Informação e Estados

É fundamental distinguir quatro níveis:

- **relato individual:** uma ocorrência submetida por um utilizador;
- **ocorrência com vários relatos semelhantes:** relatos do mesmo bairro e serviço dentro da janela temporal;
- **alerta consolidado pela plataforma:** estado atribuído quando o número de utilizadores distintos atinge o limiar definido;
- **interrupção oficialmente confirmada:** só existe se a entidade responsável a confirmar. O sistema **não** atribui automaticamente este nível.

**Tabela 7 – Estados propostos para o alerta**

| Estado | Significado |
|---|---|
| Sem alerta | Não há relatos recentes suficientes no bairro. |
| Em observação | Existem relatos, mas o número de utilizadores distintos é inferior ao limiar. |
| Alerta consolidado pela comunidade | O número de utilizadores distintos atingiu o limiar dentro da janela. Não é confirmação oficial. |
| Resolvido | Marcado por administrador ou por regra definida de normalização. |
| Expirado | Sem novos relatos durante o período de expiração definido. |

Nota: o estado «Alerta Crítico» da versão anterior foi renomeado para evitar a ideia de confirmação oficial. O estado «Manutenção Planeada» só deve ser usado se um administrador o registar com base num aviso oficial.

## 11.2. Procedimento Proposto

O procedimento considera o bairro, o tipo de serviço, a data e hora, o número de utilizadores distintos e uma janela temporal móvel. Como ponto de partida usa-se uma janela de **120 minutos**; o limiar de relatos é um **parâmetro configurável** (por exemplo, `LIMIAR`), sujeito a validação, e não uma regra universal. Se o algoritmo não estiver implementado, o procedimento é uma proposta técnica, apresentada em pseudocódigo:

```
ao receber ocorrencia(utilizador, bairro, tipo):
  se existe ocorrencia do mesmo utilizador, bairro e tipo
     dentro da JANELA: rejeitar (duplicado)
  gravar ocorrencia com data_hora actual
  n = contar utilizadores distintos com ocorrencias
      do mesmo bairro e tipo na JANELA (120 min)
  se n >= LIMIAR: estado = "Alerta consolidado pela comunidade"
  senao se n >= 1: estado = "Em observação"
  actualizar alerta (estado, ultima_actualizacao, n)

periodicamente:
  para cada alerta sem novos relatos há mais de EXPIRACAO:
     estado = "Expirado"
```

### 11.3. Parâmetros e Limitações

Os parâmetros (janela, limiar e tempo de expiração) ficam na tabela `parametro_sistema` e devem ser ajustados à dimensão de cada bairro após testes. Este mecanismo reduz o efeito de relatos isolados, mas não impede ataques coordenados nem substitui a confirmação oficial. Não se afirma que a consolidação funcione em tempo real, porque não existem provas de implementação e de testes.

## 12. Segurança e Protecção dos Dados

Descrevem-se as medidas **previstas**. A sua implementação deve ser confirmada no código e nos testes de segurança básica.

- **validação no servidor:** todos os dados recebidos são validados e normalizados, independentemente da validação feita no navegador;
- **consultas parametrizadas com PDO:** reduzem o risco de SQL Injection (OWASP, 2021);
- **palavras-passe:** devem ser guardadas com `password_hash()` e verificadas com `password_verify()` (PHP Group, s.d.). Não se recomenda SHA-256 simples como método autónomo para este fim;
- **saída segura (XSS):** os dados apresentados em páginas HTML devem ser escapados (por exemplo, com `htmlspecialchars()`). A função `filter_var` serve para validar formatos e não substitui o escape;
- **sessões:** regeneração do identificador após o início de sessão, encerramento correcto e, em produção, cookies com os atributos `HttpOnly` e `Secure`;
- **controlo de acesso:** as operações administrativas só são acessíveis ao perfil de administrador;
- **HTTPS:** obrigatório em ambiente de produção;
- **minimização de dados:** recolher apenas nome, e-mail, palavra-passe e bairro, e não mostrar dados pessoais no painel público;
- **cópias de segurança e registo de operações relevantes.**

A validação de dados não equivale a segurança completa. Nos exemplos e testes usam-se dados fictícios. Em caso de tratamento de dados pessoais reais, devem ser consideradas as obrigações da legislação angolana de protecção de dados pessoais (Lei n.º 22/11, de 17 de Junho).

## 13. Comparação Crítica das Ferramentas Computacionais

### 13.1. MySQL, PostgreSQL e Bases de Dados NoSQL

Os dados do sistema (bairros, utilizadores, ocorrências e estados) são estruturados e têm relações bem definidas, o que favorece um modelo relacional com integridade referencial e transacções ACID (Elmasri & Navathe, 2011). O MySQL é adequado por ser livre, amplamente documentado e disponível na maioria dos alojamentos web. O PostgreSQL é uma alternativa relacional robusta, com recursos geográficos avançados (extensão PostGIS), que seria útil numa evolução com mapas. Bases NoSQL (como o MongoDB) seriam mais indicadas para dados pouco estruturados ou de grande escala, o que não é o caso do protótipo.

### 13.2. PHP Nativo e Frameworks PHP

Frameworks como o Laravel oferecem segurança integrada, organização do código e ferramentas de migração, mas aumentam a complexidade e a curva de aprendizagem. Para um protótipo académico de âmbito limitado, o PHP nativo com PDO é suficiente, desde que o código seja bem organizado. Numa evolução para produção, a adopção de uma framework poderá justificar-se.

### 13.3. CSS, JavaScript e Servidor Web

O CSS nativo (Flexbox e Grid) dispensa dependências e reduz o volume de dados transferido, enquanto o Bootstrap acelera o desenvolvimento e padroniza os componentes. O JavaScript puro é suficiente para as interacções previstas; bibliotecas ou frameworks (como o React) só se justificam em aplicações mais complexas. Quanto ao servidor, o Apache é uma opção consolidada e compatível com o PHP; o Nginx é uma alternativa com bom desempenho. A escolha final deve basear-se na adequação ao âmbito e aos recursos, e não na aparente modernidade da tecnologia.

## 14. Plano de Testes do Protótipo

Os testes serão realizados com dados fictícios. Como não existe protótipo executável anexado a esta versão, os resultados observados constam como «Não executado».

**Tabela 8 – Casos de teste**

| ID | Funcionalidade | Dados de entrada | Resultado esperado | Resultado observado |
|---|---|---|---|---|
| T-01 | Registo (RF-001) | Dados válidos fictícios | Conta criada | Não executado |
| T-02 | Registo (RF-001) | E-mail já existente | Registo rejeitado | Não executado |
| T-03 | Autenticação (RF-002) | Credenciais correctas | Sessão iniciada | Não executado |
| T-04 | Autenticação (RF-002) | Palavra-passe errada | Acesso negado | Não executado |
| T-05 | Ocorrência (RF-005, RF-006) | Bairro e tipo válidos | Ocorrência gravada com data e hora | Não executado |
| T-06 | Duplicação (RF-010) | Mesmo utilizador, bairro e tipo, dentro da janela | Segundo relato rejeitado ou sinalizado | Não executado |
| T-07 | Consolidação (RF-009) | Relatos de utilizadores distintos até ao limiar | Estado passa a alerta consolidado | Não executado |
| T-08 | SQL Injection (RNF-001) | Texto malicioso nos campos | Entrada tratada como texto; sem erro de SQL | Não executado |
| T-09 | XSS (RNF-001) | Script em campo de texto | Conteúdo escapado na saída | Não executado |
| T-10 | Controlo de acesso (RNF-001) | Utilizador comum acede à área admin | Acesso negado | Não executado |
| T-11 | Painel público (RF-008) | Consulta sem sessão | Estados visíveis sem dados pessoais | Não executado |
| T-12 | Usabilidade e responsividade (RNF-004, RNF-005) | Telemóvel e computador | Interface utilizável nos dois | Pendente de validação |

# 15. Conclusões e Considerações Finais

## 15.1. Contribuições Principais

O trabalho partiu da pergunta sobre a forma como um sistema web colaborativo de alertas por bairro pode melhorar a organização, a consulta e a divulgação de informações sobre interrupções de água e de energia eléctrica em Luanda. Como resposta, foram analisado o problema, especificados requisitos funcionais e não funcionais, seleccionadas tecnologias de código aberto, propostas uma arquitectura em três camadas e uma base de dados relacional, e definida uma lógica de consolidação de relatos com parâmetros configuráveis.

Em relação aos objectivos específicos, a revisão bibliográfica, a especificação de requisitos, a análise do problema, a modelação dos dados, a comparação tecnológica e o plano de testes estão documentados. O desenvolvimento das interfaces e da consolidação, bem como a execução dos testes, dependem da implementação do protótipo. Assim, a contribuição actual é de **concepção e modelação**, e o benefício do sistema para a comunidade é uma expectativa que carece de demonstração.

## 15.2. Limitações

- não existem evidências de implementação, publicação ou testes com utilizadores;
- não foram realizadas entrevistas, inquéritos ou recolha de dados no terreno;
- o sistema depende da participação honesta da comunidade e pode ser alvo de relatos falsos;
- o alerta consolidado não é uma confirmação oficial da EPAL ou da ENDE;
- alguns dados bibliográficos devem ser confirmados no exemplar consultado.

## 15.3. Trabalhos Futuros

Como possibilidades de evolução, e não como funcionalidades concluídas, apontam-se: a implementação e validação do protótipo com utilizadores; a melhoria dos mecanismos antifraude; a integração com mapas (por exemplo, OpenStreetMap com Leaflet), tendo em conta a privacidade da localização; o envio de notificações; os relatórios estatísticos (RF-012); e mecanismos de confirmação por fontes oficiais, se as entidades responsáveis o permitirem.

# 16. Referências Bibliográficas

APACHE SOFTWARE FOUNDATION. Apache HTTP Server Documentation. Disponível em: https://httpd.apache.org/docs/. Acesso em: 2026.

CASTELLS, Manuel. A Sociedade em Rede. São Paulo: Paz e Terra, 2011. (A Era da Informação: Economia, Sociedade e Cultura, v. 1).

ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de Banco de Dados. 6.ª ed. São Paulo: Pearson Education, 2011.

FLANAGAN, David. JavaScript: O Guia Definitivo. 6.ª ed. Porto Alegre: Bookman, 2012.

GOODCHILD, Michael F. Citizens as sensors: the world of volunteered geography. GeoJournal, v. 69, n. 4, p. 211-221, 2007.

HOWE, Jeff. Crowdsourcing: Why the Power of the Crowd Is Driving the Future of Business. New York: Crown Business, 2008.

MDN WEB DOCS. Fetch API. Disponível em: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API. Acesso em: 2026.

ORACLE. MySQL Reference Manual. Disponível em: https://dev.mysql.com/doc/. Acesso em: 2026.

OWASP FOUNDATION. OWASP Top 10:2021. Disponível em: https://owasp.org/Top10/. Acesso em: 2026.

PHP GROUP. PHP Manual: PDO e password_hash. Disponível em: https://www.php.net/manual/. Acesso em: 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: Uma Abordagem Profissional. 8.ª ed. Porto Alegre: AMGH, 2016.

REPÚBLICA DE ANGOLA. Lei n.º 22/11, de 17 de Junho: Lei da Protecção de Dados Pessoais. Diário da República.

SOMMERVILLE, Ian. Engenharia de Software. 9.ª ed. São Paulo: Pearson Prentice Hall, 2011.

USHAHIDI. Ushahidi Platform. Disponível em: https://www.ushahidi.com. Acesso em: 2026.

W3C. HTML5: A vocabulary and associated APIs for HTML and XHTML. W3C Recommendation, 28 out. 2014. Disponível em: https://www.w3.org/TR/html5/. Acesso em: 2026.